

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

23DSA203 / 23CSE203: Data Structures and Algorithms

Topic: **Linked List** (10 Challenge Programs) | Roll No: **CH.SC.U4CSE25262**

Question 1: Delete All Occurrences of a Key in Linked List

Problem: Given a singly linked list and a key value, delete all nodes containing that key and display the updated linked list.

Sample I/O: Input: 6 \n 7 9 8 8 4 2 \n 8 | Output: Linked List:->7->9->4->2

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

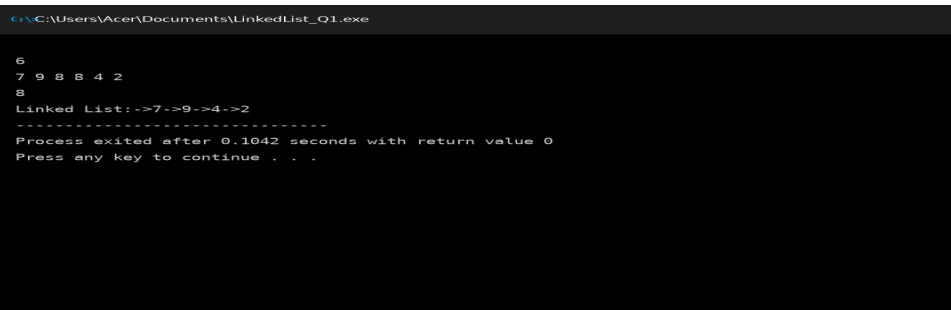
void create(int val) {
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = val;
    new_node->next = NULL;
    if (start == NULL) {
        start = new_node;
        return;
    }
    struct node *temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

void del(int key) {
    while (start != NULL && start->data == key) {
        struct node *temp = start;
        start = start->next;
        free(temp);
    }
    struct node *p1 = start;
    while (p1 != NULL && p1->next != NULL) {
        if (p1->next->data == key) {
            struct node *p2 = p1->next;
            p1->next = p2->next;
            free(p2);
        } else {
            p1 = p1->next;
        }
    }
}

void display() {
    printf("Linked List:");
    struct node *temp = start;
    while (temp != NULL) {
        printf("->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        start = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            create(val);
        }
        int key;
        scanf("%d", &key);
        del(key);
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\LinkedList_Q1.exe
6
7 9 8 8 4 2
8
Linked List:->7->9->4->2
-----
Process exited after 0.1042 seconds with return value 0
Press any key to continue . . .
```

Question 2: Doubly Linked List Reverse and Forward Display

Problem: Given a doubly linked list constructed by inserting nodes at the beginning, display the list from head to tail and then traverse backward using prev pointers.

Sample I/O: Input: 16 \n 1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17 | Output: Forward and Reverse sequence of elements

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *prev;
};

void insertStart(struct Node** head, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = *head;
    new_node->prev = NULL;
    if (*head != NULL) {
        (*head)->prev = new_node;
    }
    *head = new_node;
}

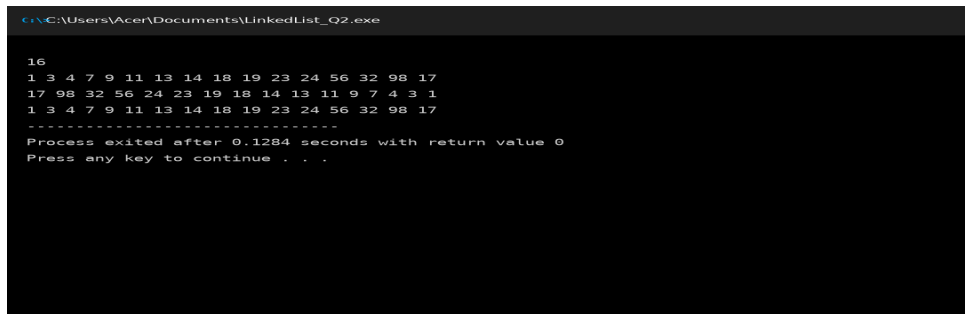
void printReverseAndForward(struct Node* head) {
    if (head == NULL) return;
    struct Node* temp = head;
    struct Node* last = NULL;

    // Print reverse (since we inserted at start, head-to-tail is reverse of input, or we traverse to end)
    while (temp != NULL) {
        printf("%d ", temp->data);
        if (temp->next == NULL) last = temp;
        temp = temp->next;
    }
    printf("\n");

    // Print forward using prev pointers
    temp = last;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        struct Node* head = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            insertStart(&head, val);
        }
        printReverseAndForward(head);
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\LinkedList_Q2.exe

16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
17 98 32 56 24 23 19 18 14 13 11 9 7 4 3 1
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
-----
Process exited after 0.1284 seconds with return value 0
Press any key to continue . . .
```

Question 3: Sorted Insert in Linked List

Problem: Given a sequence of integers, maintain a sorted linked list by inserting each element at its correct sorted position using sortedInsert.

Sample I/O: Input: 6 \n 10 20 30 45 34 56 | Output: 10 20 30 34 45 56

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void sortedInsert(struct Node** head_ref, struct Node* new_node) {
    struct Node* current;
    if (*head_ref == NULL || (*head_ref)->data >= new_node->data) {
        new_node->next = *head_ref;
        *head_ref = new_node;
    } else {
        current = *head_ref;
        while (current->next != NULL && current->next->data < new_node->data) {
            current = current->next;
        }
        new_node->next = current->next;
        current->next = new_node;
    }
}

struct Node* newNode(int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;
    return new_node;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        struct Node* head = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            sortedInsert(&head, newNode(val));
        }
        printList(head);
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\LinkedList_Q3.exe
6
10 20 30 45 34 56
10 20 30 34 45 56
-----
Process exited after 0.0985 seconds with return value 0
Press any key to continue . . .
```

Question 4: Insert Before a Given Key in Linked List

Problem: Given a linked list, insert a new node with value val immediately before the first occurrence of a given key node. If key is not found, print 'Node not found!'.

Sample I/O: Input: 4 \n 115 210 116 110 \n 116 \n 17 | Output: Linked List:->115->210->17->116->110

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

void create(int val) {
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = val;
    new_node->next = NULL;
    if (start == NULL) {
        start = new_node;
        return;
    }
    struct node *temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

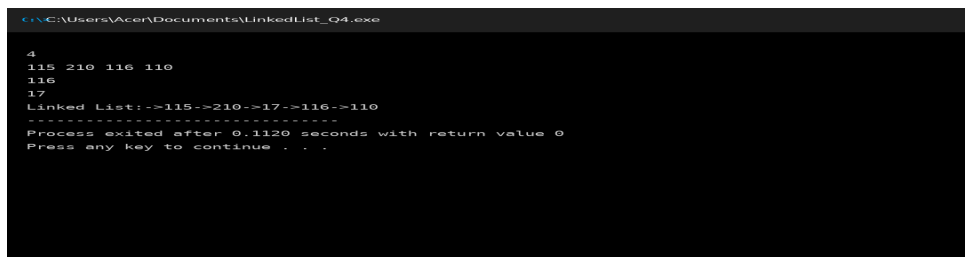
void insertBefore(int key, int val) {
    struct node *p1 = (struct node*)malloc(sizeof(struct node));
    p1->data = val;
    p1->next = NULL;

    if (start == NULL) {
        printf("Node not found!\n");
        return;
    }
    if (start->data == key) {
        p1->next = start;
        start = p1;
        return;
    }
    struct node *p2 = start;
    while (p2->next != NULL && p2->next->data != key) {
        p2 = p2->next;
    }
    if (p2->next == NULL) {
        printf("Node not found!\n");
        free(p1);
    } else {
        p1->next = p2->next;
        p2->next = p1;
    }
}

void display() {
    printf("Linked List:");
    struct node *temp = start;
    while (temp != NULL) {
        printf("->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        start = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            create(val);
        }
        int key, val;
        scanf("%d %d", &key, &val);
        insertBefore(key, val);
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\LinkedList_Q4.exe
4
115 210 116 110
116
17
Linked List:->115->210->17->116->110
-----
Process exited after 0.1120 seconds with return value 0
Press any key to continue . . .
```

Question 5: Delete Alternate Nodes in Linked List

Problem: Construct a linked list with integers from 1 to N, and delete every alternate node starting from the second node.

Sample I/O: Input: 10 | Output: 1 3 5 7 9

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void insert_Data(struct node **head, int val) {
    struct node* new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = val;
    new_node->next = NULL;
    if (*head == NULL) {
        *head = new_node;
        return;
    }
    struct node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

void delete_Alt(struct node **head) {
    if (*head == NULL) return;
    struct node *prev = *head;
    struct node *curr = (*head)->next;

    while (prev != NULL && curr != NULL) {
        prev->next = curr->next;
        free(curr);
        prev = prev->next;
        if (prev != NULL) {
            curr = prev->next;
        }
    }
}

void printList(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        struct node *head = NULL;
        for (int i = 1; i <= n; i++) {
            insert_Data(&head, i);
        }
        delete_Alt(&head);
        printList(head);
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\LinkedList_Q5.exe

10
1 3 5 7 9
-----
Process exited after 0.0895 seconds with return value 0
Press any key to continue . . .
```

Question 6: Get Nth Node in Reverse Linked List

Problem: Construct a linked list by inserting elements at the head, display the resulting linked list, and return the data value at the given 1-based index.

Sample I/O: Input: 4 \n 1 2 3 4 \n 2 | Output: Linked list:-->4-->3-->2-->1 \n Node at index=2:3

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void push(struct node** head_ref, int new_data) {
    struct node* new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = new_data;
    new_node->next = *head_ref;
    *head_ref = new_node;
}

int GetNth(struct node* head, int index) {
    struct node* current = head;
    int count = 1;
    while (current != NULL) {
        if (count == index)
            return current->data;
        count++;
        current = current->next;
    }
    return -1;
}

void display(struct node* head) {
    printf("Linked list:");
    struct node* temp = head;
    while (temp != NULL) {
        printf("-->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        struct node* head = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            push(&head, val);
        }
        int idx;
        scanf("%d", &idx);
        display(head);
        printf("Node at index=%d:%d\n", idx, GetNth(head, idx));
    }
    return 0;
}
```

Execution Output:

```
C:\Users\Acer\Documents\LinkedList_Q6.exe
4
1 2 3 4
2
Linked list:-->4-->3-->2-->1
Node at index=2:3
-----
Process exited after 0.1190 seconds with return value 0
Press any key to continue . . .
```

Question 7: Delete All Elements Before a Given Key

Problem: Given a linked list and a key value, delete all nodes preceding the key so that the key becomes the new head. If key does not exist, print 'Invalid Node!'.

Sample I/O: Input: 4 \n 9 77 12 6 \n 12 | Output: Linked List:->12->6

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

void create(int val) {
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = val;
    new_node->next = NULL;
    if (start == NULL) {
        start = new_node;
        return;
    }
    struct node *temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

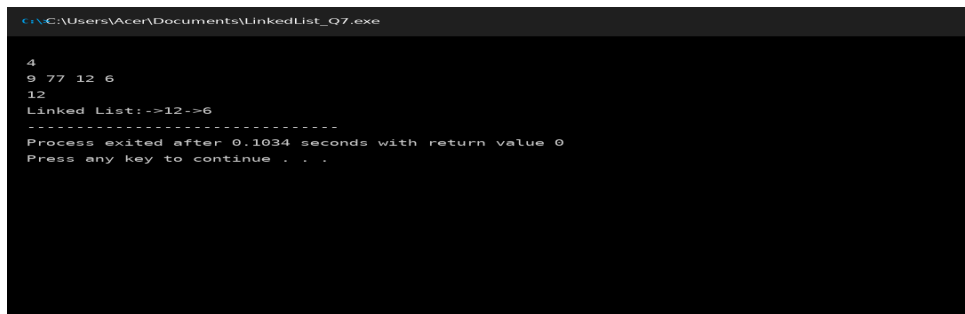
void del(int key) {
    struct node *p1 = start;
    int found = 0;
    while (p1 != NULL) {
        if (p1->data == key) {
            found = 1;
            break;
        }
        p1 = p1->next;
    }

    if (!found) {
        printf("Invalid Node! ");
    } else {
        while (start != NULL && start->data != key) {
            struct node *temp = start;
            start = start->next;
            free(temp);
        }
    }
}

void display() {
    printf("Linked List:");
    struct node *temp = start;
    while (temp != NULL) {
        printf("->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        start = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            create(val);
        }
        int key;
        scanf("%d", &key);
        del(key);
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\LinkedList_Q7.exe
4
9 77 12 6
12
Linked List:->12->6
-----
Process exited after 0.1034 seconds with return value 0
Press any key to continue . . .
```

Question 8: Fold Linked List (In-Place Interleaving)

Problem: Given a linked list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$, reorder it to the folded pattern $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \dots$

Sample I/O: Input: 16 \n 1 3 4 ... 17 | Output: Link list data after fold: 1 17 3 98 4 32 ...

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void create(struct node **head, int data) {
    struct node* new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = NULL;
    if (*head == NULL) {
        *head = new_node;
        return;
    }
    struct node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

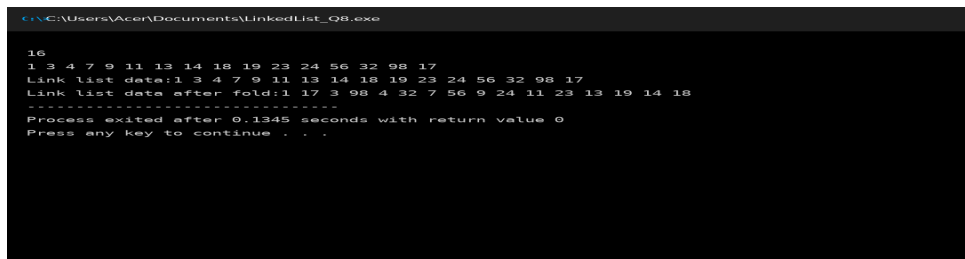
void print(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void fold(struct node *head) {
    int arr[100];
    int n = 0;
    struct node *temp = head;
    while (temp != NULL) {
        arr[n++] = temp->data;
        temp = temp->next;
    }

    int left = 0, right = n - 1;
    printf("Link list data after fold:");
    while (left <= right) {
        if (left == right) {
            printf("%d ", arr[left]);
        } else {
            printf("%d %d ", arr[left], arr[right]);
        }
        left++;
        right--;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        struct node *head = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            create(&head, val);
        }
        printf("Link list data:");
        print(head);
        fold(head);
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\LinkedList_Q8.exe
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18
Process exited after 0.1345 seconds with return value 0
Press any key to continue . . .
```

Question 9: Circular Linked List Odd-Even Split

Problem: Construct a circular linked list of size n, display the complete circular list, and then display two circular sub-lists containing odd and even numbers respectively.

Sample I/O: Input: 6 | Output: Complete linked_list, Odd sub-list, Even sub-list

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct n {
    int data;
    struct n *next;
};

struct n* head = NULL;

void insert(int data) {
    struct n* new_node = (struct n*)malloc(sizeof(struct n));
    new_node->data = data;
    new_node->next = NULL;

    if (head == NULL) {
        head = new_node;
        head->next = head;
        return;
    }
    struct n* temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    temp->next = new_node;
    new_node->next = head;
}

void display(struct n *h) {
    if (h == NULL) return;
    printf("[h]");
    struct n* temp = h;
    do {
        printf("=>%d", temp->data);
        temp = temp->next;
    } while (temp != h);
    printf("=>[h]\n");
}

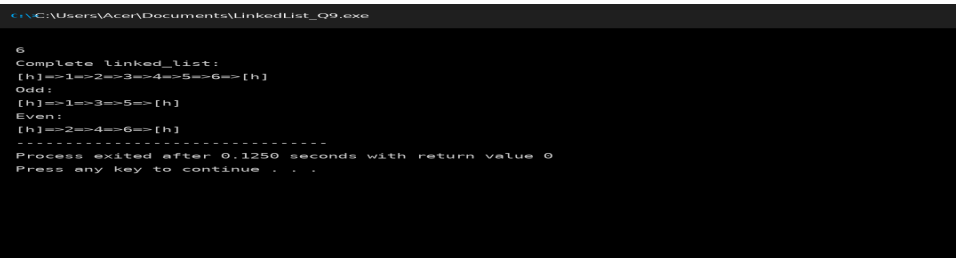
void displayOddEven(int n) {
    printf("Complete linked_list:\n");
    display(head);

    printf("Odd:\n[h]");
    for (int i = 1; i <= n; i += 2) {
        printf("=>%d", i);
    }
    printf("=>[h]\n");

    printf("Even:\n[h]");
    for (int i = 2; i <= n; i += 2) {
        printf("=>%d", i);
    }
    printf("=>[h]\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        head = NULL;
        for (int i = 1; i <= n; i++) {
            insert(i);
        }
        displayOddEven(n);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\LinkedList_Q9.exe
6
Complete linked_list:
[h]=>1=>2=>3=>4=>5=>6=>[h]
Odd:
[h]=>1=>3=>5=>[h]
Even:
[h]=>2=>4=>6=>[h]
Process exited after 0.1250 seconds with return value 0
Press any key to continue . . .
```

Question 10: Delete First K Nodes in Linked List

Problem: Given a singly linked list and an integer k, remove the first k nodes from the head of the list and display the remaining list.

Sample I/O: Input: 4 \n 9 77 12 6 \n 2 | Output: Linked List:->12->6

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

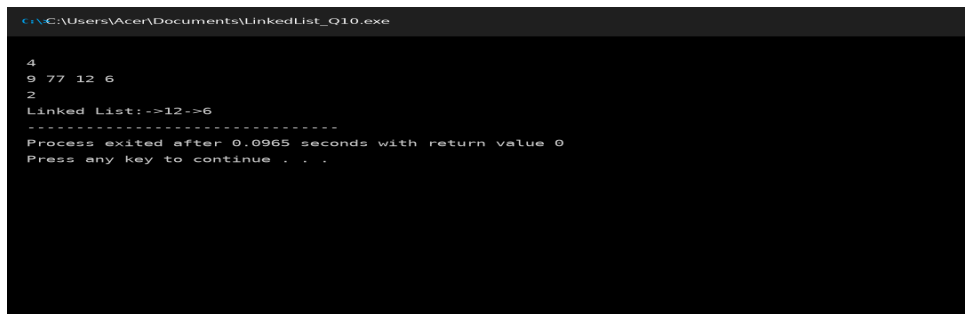
void create(int val) {
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = val;
    new_node->next = NULL;
    if (start == NULL) {
        start = new_node;
        return;
    }
    struct node *temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

void delFirstK(int k) {
    for (int i = 0; i < k && start != NULL; i++) {
        struct node *temp = start;
        start = start->next;
        free(temp);
    }
}

void display() {
    printf("Linked List:");
    struct node *temp = start;
    while (temp != NULL) {
        printf("->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        start = NULL;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            create(val);
        }
        int k;
        scanf("%d", &k);
        delFirstK(k);
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\LinkedList_Q10.exe

4
9 77 12 6
2
Linked List:->12->6
-----
Process exited after 0.0965 seconds with return value 0
Press any key to continue . . .
```